

CSE4820 Final Project Report

Predicting Diabetes Diagnoses and Student Grades from Behavioral & Clinical Data

Samuel Schultz | CSE 4820 | Spring 2026

1. Introduction

This project explores two different supervised learning tasks for two very different real-world datasets. The first is the Diabetes dataset, a small clinical dataset where the task is binary classification: given eight medical measurements, predict whether a patient has diabetes. The second is a much larger Gaming and Academic Performance dataset, where the task is regression: given a student's gaming, sleep, study, and lifestyle features, predict their numeric course grade. The two datasets pull in opposite directions. The diabetes data is small and noisy with lots of disguised missing values, while the gaming data is large, clean, and easy to model. This makes them a useful pair for seeing where each technique helps.

2. Datasets and Problem Definition

2.1 Pima Indians Diabetes Dataset

The diabetes dataset contains 768 patient records with 8 numeric features and a binary outcome label (0 = no diabetes, 1 = diabetes). The features are: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, and Age. About 35% of patients in the dataset are positive cases, which is imbalanced but not severely so. standard classifiers can handle this without resampling.

The catch with this dataset is that several columns use 0 as a placeholder for missing measurements. A glucose level of 0 or a BMI of 0 isn't physically possible; these are missing values that were encoded as zeros. Insulin had 374 such zeros (nearly half the dataset), SkinThickness had 227, and BloodPressure had 35. The first preprocessing step was to replace these with NaN so the imputer could handle them properly later in the pipeline.

Problem: Binary classification: given the 8 clinical measurements, predict whether the patient has diabetes. I used ROC AUC as the primary tuning metric since the classes are imbalanced and AUC is threshold-independent.

2.2 Gaming and Academic Performance Dataset

The gaming dataset contains 8,000 student records with 13 features and a continuous numeric grade (0-100) as the target. Features include gaming_hours, study_hours, sleep_hours, attendance, device_usage, reaction_time_ms, addiction_score, social_activity, age, and categorical fields for gender, gaming_genre, and stress_level. The dataset has no missing values.

Problem: Regression: predict the student's numeric grade from their lifestyle and gaming features. I used mean absolute error (MAE) as the primary tuning metric, with RMSE and R^2 reported alongside, and added a derived letter-grade classification analysis at the end to get a more interpretable view of accuracy.

3. Methodology

3.1 Splitting and Preprocessing — Diabetes

I used an 80/20 train/test split stratified on Outcome, then split the training set again 80/20 to carve out a validation set. After replacing the impossible zeros with NaN, I built three candidate preprocessing pipelines so the grid search could pick the best one rather than me guessing:

- Pipeline 1 (standard): median imputation followed by standard scaling.
- Pipeline 2 (interactions): same as above, plus engineered features BMI^2 and $BMI \times Glucose$. The BMI–glucose interaction is clinically meaningful because elevated BMI drives insulin resistance, which in turn pushes blood glucose higher. So a patient who is high on both axes is in a different risk category than one who is high on only one, and a model that sees only the linear features can't represent that interaction directly.
- Pipeline 3 (polynomial): impute, scale, then a degree-2 polynomial expansion of all numeric features.

Median imputation was chosen over the mean because Insulin is heavily right-skewed and the median is much more robust to that. The exploratory plots below confirm the skew.

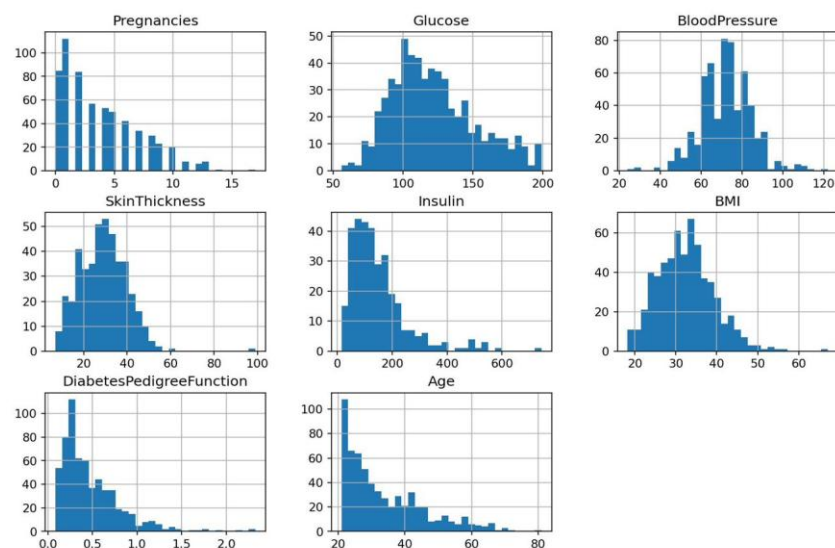


Figure 1. Feature histograms after replacing impossible zeros with NaN. Glucose and BMI are roughly normal; Insulin, Age, and Pregnancies are right-skewed.

The correlation heatmap made the picture even clearer: Glucose is by far the strongest predictor of Outcome, with BMI and Age trailing as secondary signals. Insulin and SkinThickness, despite being clinically meaningful, barely correlate with the outcome, I would presume because of how much data is missing for both.

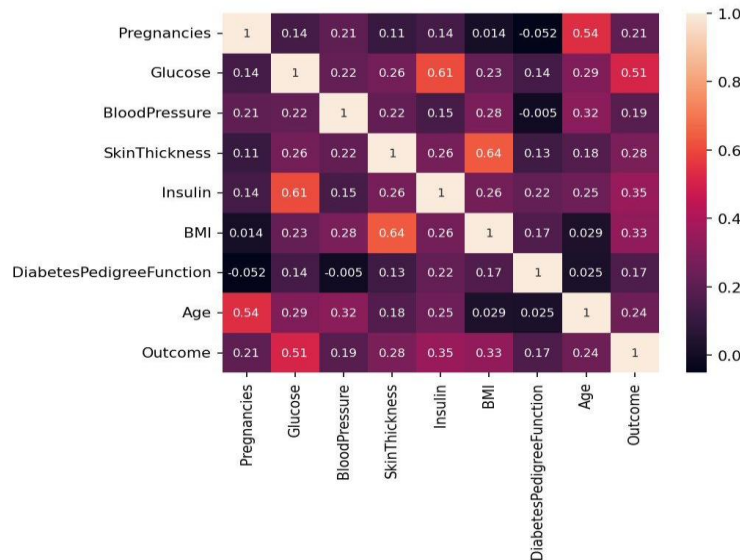


Figure 2. Correlation heatmap. Glucose dominates as a predictor of Outcome.

3.2 Splitting and Preprocessing — Gaming

Same nested-split strategy as the diabetes data: 80/20 train/test with letter-grade stratification, then another 80/20 split on the training set for validation. Letter grades (A–F) were derived from the numeric grade just for the purpose of stratification — the model still trains on the continuous target.

During exploration I found that `student_id`, `age`, `social_activity`, `gaming_genre`, added no predictive value. They were dropped before training. The clustermap below shows the underlying relationships among the numeric features and was a big part of that decision.

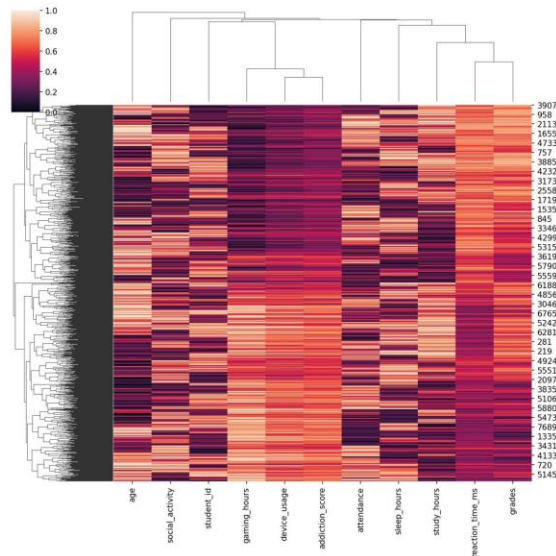


Figure 3. Clustermap of numeric features. Reaction time clusters closest to grades; gaming_hours, device_usage, and addiction_score form a tight 'gamer profile' cluster.

Three preprocessing pipelines were defined, mirroring the structure used for the diabetes problem:

- Pipeline 1 (standard): standard scaling for numeric features, ordinal encoding for stress_level (Low → Medium → High).
- Pipeline 2 (composite): collapse the four highly correlated 'gamer' features (gaming_hours, device_usage, reaction_time_ms, addiction_score) into a single composite 'gamer_score', flipping reaction_time so all four point in the same direction.
- Pipeline 3 (polynomial): degree-2 polynomial expansion of all numeric features.

The composite feature alone correlated -0.53 with grades, which is a strong signal in a single column. Whether collapsing four features into one helps a model versus just letting it find the same structure on its own was something I let the cross-validation decide.

3.3 Models — Diabetes (Classification)

Three model families were tuned with 5-fold cross-validation, scoring on ROC AUC:

- Logistic Regression: GridSearchCV over penalty (l1, l2, elasticnet), C, and l1_ratio. 90 candidates $\times$ 5 folds = 450 fits.
- Support Vector Classifier: RandomizedSearchCV over rbf, linear, and poly kernels, with C and gamma sampled log-uniformly. 50 iterations $\times$ 5 folds = 250 fits.
- Histogram Gradient Boosting: RandomizedSearchCV over learning_rate, max_iter, max_depth, min_samples_leaf, l2_regularization, max_leaf_nodes. 60 iterations $\times$ 5 folds = 300 fits.

Each search was given all three preprocessing pipelines as a hyperparameter, so the grid effectively picks the best preprocessor and model jointly rather than fixing one and tuning the other.

3.3.1 Model Selection Rationale

Logistic Regression was included as a strong, well-understood linear baseline. It acts functions as a base line model for all subsequent models to compare themselves to. Support Vector Classification was chosen to test whether a non-linear decision boundary gives us anything on this dataset. It uses the kernel trick to project the data into a higher-dimensional feature space where classes that overlap in the original eight features may become separable. Histogram Gradient Boosting was added as the non-parametric, tree-based option. Together, the three models cover three very different inductive biases —linear, kernel-based, and tree-based — so whichever one wins also tells me something about what kind of structure the data has.

3.4 Models — Gaming (Regression)

Seven model families were tuned, each scored with 5-fold CV on negative MAE:

- SGDRegressor: full grid over loss (squared_error, huber, epsilon_insensitive) and penalty (l1, l2, elasticnet, none).
- Support Vector Regressor: randomized search over rbf, linear, polynomial (degrees 2–4), and sigmoid kernels.
- Kernel Ridge Regression: randomized search over alpha and gamma with rbf kernel.
- Random Forest Regressor: randomized search over n_estimators, max_depth, min_samples_split/leaf, and max_features.
- MLP Regressor: randomized search over hidden_layer_sizes, activation (relu, tanh), alpha, and learning_rate_init.
- KNN Regressor: randomized search over n_neighbors, weights, and Manhattan/Euclidean distance.
- HistGradientBoosting (with Nyström kernel approximation): final boosting baseline with rbf/poly Nyström features.

All searches used the same three preprocessing pipelines as candidate hyperparameters. This was computationally heavier than the diabetes setup, the MLP search alone took about 5 minutes.

3.4.1 Model Selection Rationale

Model selection happened in two phases. The first phase was a screening pass with three models spanning very different inductive biases. SGDRegressor was the linear baseline, it's flexible enough to behave like ordinary least squares, Lasso, Ridge, or Huber regression

depending on the loss and penalty, so cross-validating across that grid effectively asks which linear formulation suits the data best. SVR was chosen next because the EDA showed the relationship between several features and grades was approximately but not perfectly linear, and SVR's kernel options give a clean way to test whether a modest amount of non-linearity helps. Histogram Gradient Boosting was the third choice, a tree-based model that can capture arbitrary non-linear interactions without any explicit feature engineering.

The screening results pointed clearly toward non-linear structure: SVR with the polynomial preprocessing and an RBF kernel, and HGB with polynomial preprocessing, were the strongest performers. That motivated the second phase, which added four more models to push on that finding from different angles. Kernel Ridge Regression was added because it's the squared-loss analog of SVR. Its different loss function is typically more robust against noisy targets. Random Forest was included as a second tree-based option for comparison against HGB. MLPRegressor tested whether a small neural network could learn a richer representation than the kernel methods. KNN was included as a simple non-parametric option with extremely flexible decision boundaries. A sanity check that the more complex models were doing something useful.

Finally, HGB with Nyström kernel approximation was tested as a deliberate combination of the two strongest signals from phase one (boosted trees + RBF/poly kernels) to see whether stacking those advantages produced any further gain.

4. Evaluation and Comparison

4.1 Diabetes — Validation Results

All three classifiers landed in roughly the same neighborhood, which is what I'd expect from a small, well-known dataset like this. There's a ceiling on how much performance a model can squeeze out of 768 records when the dominant signal is a single feature (Glucose). The validation results were:

Model	CV AUC	Val AUC	Val Accuracy	Val F1
SVC (rbf)	0.8368	0.8727	0.8049	0.6923
Logistic Regression	0.8373	0.8712	0.7886	0.6750
Hist Gradient Boosting	0.8252	0.8555	0.7805	0.6824

SVC with an RBF kernel came out narrowly on top with a validation AUC of 0.8727 and the highest accuracy. The boosted trees did slightly worse, which I think makes sense: HGB usually

shines on larger datasets, and with only 491 training rows it doesn't have enough data to outperform the simpler models. The ROC curves for all three are visibly overlapping.

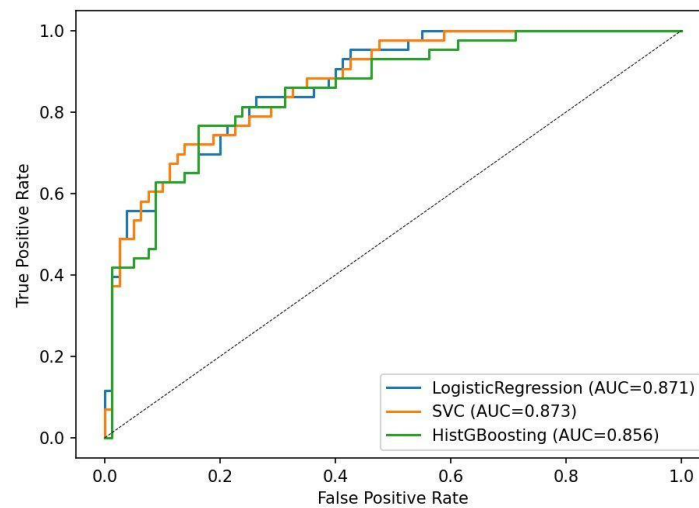


Figure 4. ROC curves on the validation set. All three models are tightly clustered, with SVC narrowly leading.

The learning curve for SVC is still trending upward at the full training size, which suggests more data would help more than additional hyperparameter tuning. That's a useful diagnostic; it tells me I'm not overfitting and not underfitting either, just data-limited.

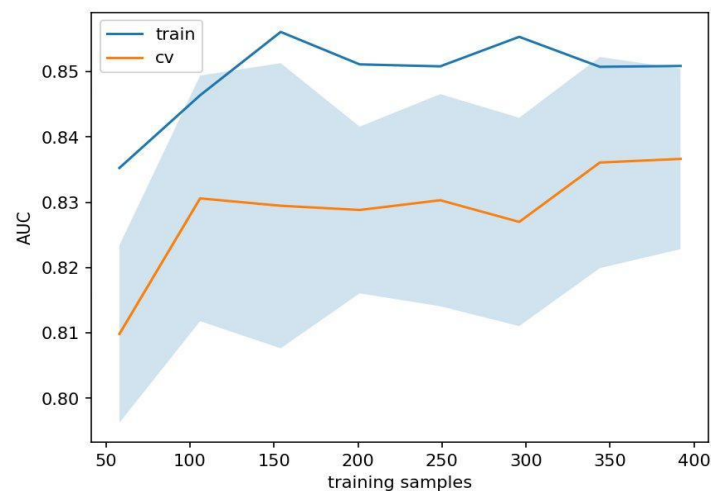


Figure 5. Learning curve for SVC. The CV AUC is still climbing as training size increases.

4.2 Diabetes — Test Set Performance

On the held-out test set, the SVC achieved approximately 72% accuracy with an AUC of 0.82. The confusion matrix shows the model catches most non-diabetic patients correctly but misses a meaningful fraction of diabetic ones. For a real screening tool this isn't ideal; false positives cause far less harm than false negatives. So, I'd recommend lowering the decision threshold from 0.5 to about 0.35–0.40 in deployment to trade some precision for higher recall.

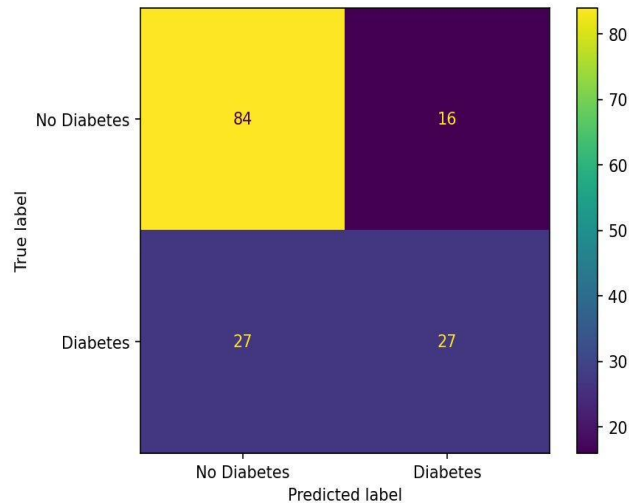


Figure 6. Test-set confusion matrix for the SVC. Final test metrics: Accuracy 0.721, F1 0.557, AUC 0.817. False negatives (top-right) are the costliest error type.

4.3 Gaming — Validation Results

All seven regressors were tuned and compared on the validation set. The top three landed remarkably close to each other, all within ~0.10 MAE:

Model	CV MAE	Val MAE	Val RMSE	Val R ²
HistGradientBoosting	4.5404	4.4921	5.7696	0.9348
MLP	4.5732	4.5729	5.8267	0.9335
Kernel Ridge (rbf)	4.5890	4.5846	5.8274	0.9334
SVR (rbf, poly preproc.)	4.5746	4.6257	5.8964	0.9319
Random Forest	4.7543	4.6896	6.0056	0.9293
SGD (linear)	4.8531	4.7870	6.1035	0.9270
KNN	5.2642	5.1707	6.5942	0.9148

Histogram Gradient Boosting came out on top at validation MAE 4.49 and R² 0.935, with MLP and Kernel Ridge close behind at MAE ≈ 4.57. The spread between the top four (HGB, MLP, KR, SVR) is only about 0.13 MAE — a remarkably tight cluster considering how different these algorithms are internally. When I correlated all the model predictions against each other on the validation set, every pairwise correlation exceeded 0.99. This implies we’ve essentially hit the ceiling of what’s learnable from this feature set. Stacking won’t help; they’ve all converged on the same underlying function. So, I dropped the planned stacking step and moved forward with

HGB as the final model as it had the best validation metrics, fast inference, and didn't need polynomial-feature expansion at test time.

4.4 Gaming — Test Set Performance

On the held-out test set, HGB achieved an MAE of 4.53, an MSE of 32.81, and an R^2 of 0.934, essentially identical to validation, which is a good sign of generalization. Predictions match the true grade within roughly ± 4.5 percentage points on average.

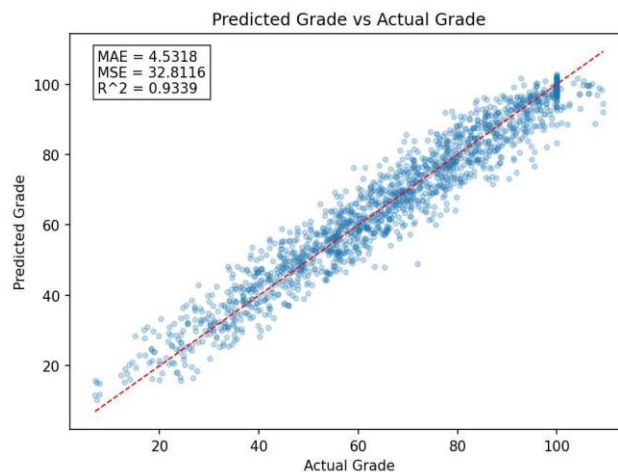


Figure 7. Predicted vs. actual grade on the test set. The fit tracks the diagonal cleanly with mild compression at the extremes.

There is a slight tendency to overpredict in the very low range (below ~ 20) and underpredict at the very top (above ~ 95), but it's small and consistent with what most regressors do near the edges of the training distribution. The R^2 of 0.934 means the model explains 93.4% of the variance in grades from these features alone, which is an unusually clean signal.

To make the result more interpretable, I converted both the actual and predicted continuous grades into letter grades (A/B/C/D/F using standard 90/80/70/60 cutoffs) and computed classification metrics. Accuracy was 0.730, weighted precision 0.733, and weighted F1 0.731.

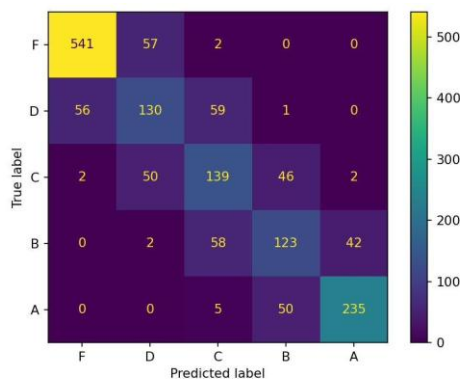


Figure 8. Letter-grade confusion matrix. Misclassifications are almost always within one letter grade of the true label.

Looking at the letter-grade confusion matrix, the failures are essentially never catastrophic: a true A is rarely predicted as anything below a B, and failing students (F) are almost always identified correctly. So while a 27% letter-grade error rate sounds high in isolation, in practice every wrong answer is usually only one letter grade off.

5. Conclusion

Two distinct supervised learning problems were tackled end-to-end. For the diabetes binary classification task, an SVC with an RBF kernel achieved 72.1% test accuracy and a test ROC AUC of 0.817, with Glucose carrying nearly all the predictive weight; the learning curve still trending upward at the full training size suggests more data, not more tuning, is the highest-leverage next step. For the gaming grade regression task, a tuned Histogram Gradient Boosting regressor achieved a test MAE of 4.53 and an R^2 of 0.934, predicting numeric grades within roughly ± 4.5 points and letter grades correctly 73% of the time (with errors essentially always within one letter grade of truth). Across both projects, the recurring lesson was the same: most of the performance comes from getting the data right. Data exploration is an often-neglected essential component in the machine learning process, and only after it's thoroughly and competently done does model selection start to matter. Once a few well-tuned models converge to the same answer, additional complexity tends to buy nothing.